

Internet Control Message Protocol

(Lab-05 09/10 October 2025)

Summary

- Introduction to the Internet Control Message Protocol (ICMP).
- Introduction to the Ping network application based on ICMP.
- Introduction to using non-blocking sockets via the *Select* system call.
- Exercise - Complete the implementation of the Pinger application.

ICMP and Ping Application

0. Introduction

The Internet Control Message Protocol (ICMP) is a protocol that end hosts and routers use to exchange network-layer information. It is most commonly used for error reporting, but it can also carry other operational information. ICMP messages are carried as a payload in IP packets with a special value in the upper-layer protocol field of the IP header. Therefore, datagrams containing ICMP packets are demultiplexed similarly to TCP or UDP packets.

Figure 1 illustrates the format of the ICMP header. ICMP messages start with 1-byte type and code fields, followed by a 2-byte checksum for error checking. Finally, they include a 4-byte section whose content and meaning vary depending on the type of ICMP message. Following the header fields, an ICMP error message for IPv4 contains a data section carrying the entire IPv4 header and the first eight bytes of the IP datagram that generated the error.

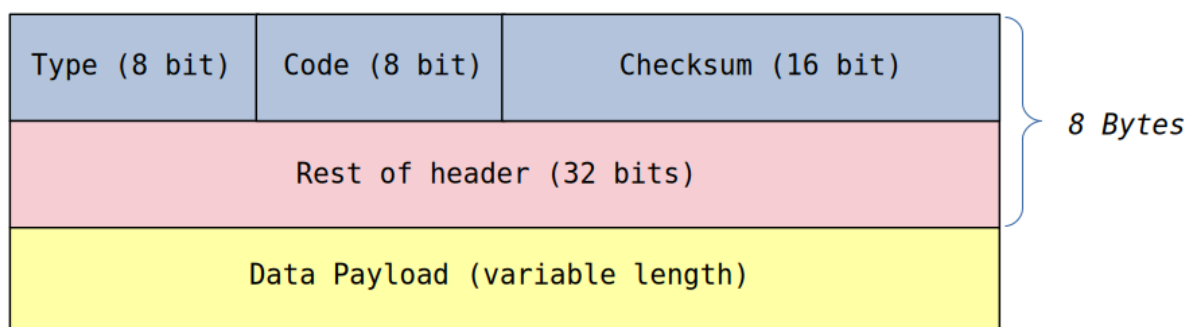


Figure 1. Header format of the Internet Control Message Protocol.

The **ping** program is a computer network application used to test if a particular host can be reached across an IP network. Ping can also be used to self-test a computer's network interface card or to perform a latency test. Ping works by sending ICMP "echo request" (type 8, code 0) packets to the target host and listening for the respective ICMP "echo reply" (type 0, code 0) packets. The "echo reply" is sometimes called a pong. Hence, the ping program is essentially a network application client that instructs the underlying operating system to generate an ICMP "echo request" message. One such client is the ping program included with your operating system's networking utilities. Most TCP/IP network stack implementations support the server side of a ping application directly within the operating system, meaning the server is not a separate application.

Figure 2 illustrates the format of the ICMP header for echo request and reply messages. The second part carries two specific fields: a 16-bit identifier and a 16-bit sequence number.

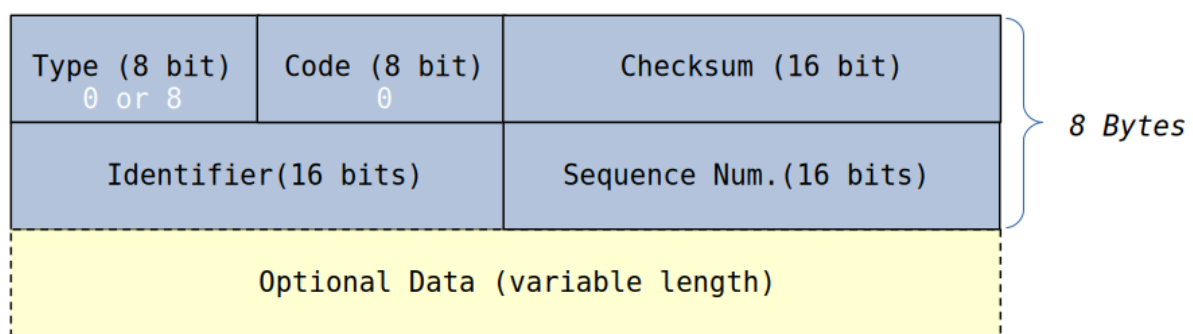


Figure 2. Header format of the Internet Control Message Protocol for Echo Request/Reply packets.

Echo Request ICMP packet

A ping is an ICMP echo request packet composed as follows:

- The *type* header field must be set to 8.
- The *code* header field must be set to 0.
- The *Identifier* and *Sequence Number* fields can be used by the client to match the reply with the request that caused the reply. In practice, most Linux systems use a unique identifier for every ping process, and sequence number is an increasing number within that process. Windows uses a fixed identifier, which varies between Windows versions, and a sequence number that is only reset at boot time.
- All data received by the echo request must be entirely included in the echo reply.

Echo Reply ICMP packet

The expected response to an echo request ("ping") is an echo reply ("pong"). The echo reply is an ICMP packet composed as follows:

- The *type* and *code* fields must be set to 0.
- The *identifier* and *sequence number* values link echo replies to their respective echo requests.
- All data received in the echo request must be entirely included in the echo reply.

Ping clients typically measure round-trip time, record packet loss, and print a statistical summary of the received echo reply packets (minimum, maximum, mean round-trip time, and, in some versions, standard deviation of the mean).

Learning materials:

- Official RFC Document defining the initial version of ICMP:
<https://www.rfc-editor.org/rfc/rfc792.html>
- The manual of the *ping* command (type *man ping* in a shell) of the *iputils* commonly available in GNU/Linux distributions, for example:
<https://manpages.debian.org/testing/iputils-ping/ping.8.en.html>
- Ping command basics for network testing and troubleshooting:
<https://www.redhat.com/en/blog/ping-usage-basics>

1. Trying out the *ping* program and observing its responses

Let's run some experiments with the ping application available on your Docker containers. To begin with, start a new container named *ping_tests_c1* with the usual image we use for our labs. This image contains the *iputils* package, which includes the ping program.

```
$docker run --name ping_tests_c1 -ti <img-name>
```

Ping-ing your local network interface

Once you are inside your container shell, launch the ping program to perform three consecutive ping attempts toward your localhost with the following command:

```
root@container_c1:$ ping -c 3 localhost
```

The *ping* program sends three consecutive ICMP echo requests and records the response times of the respective ICMP echo replies. The program concludes by printing the collected statistics. As you may realize, the response time for each packet is very short since we are testing a local network interface.

Ping-ing another docker container

Now, let's start a new container and use *ping* to confirm the reachability of this machine on the Docker network. Then, observe the measurement times.

```
$docker run --name ping_tests_c2 -ti <img-name>
```

```
root@container_c2:$ ip addr show
```

first note the IP address of the newly created container produced by the *ip* command. Then, use that address to run a *ping* from the first container:

```
root@container_c1:$ ping -c 3 IP_address_container_2
```

You should see that all of your ping echo packets have been replied to, resulting in 0% packet loss. This indicates that the second container is accessible via the network, and the network is functioning

properly. As you may have noticed, the ping response times are still very low, which is expected since the containers are running on the same machine, similar to the previous ping experiment.

Ping-ing an external host on the Internet

To get an idea of the possible response times over the public internet, let's now ping an external host.¹

```
root@container_id:$ ping -c 3 google.pt
```

You should realize that the average response time to ping packets has increased in this test. This is due to the packets traveling to and from your host machine through the public internet. As you should have learned in class, the average round-trip time depends on various factors, such as network conditions and the geographical position of the targeted host.

2. Program Example

In the lab05_files.zip, we have provided you with two basic client and server programs that demonstrate how to send and receive ICMP packets through raw sockets. The client program *send_ping.py* manually crafts and sends ICMP echo request packets through raw sockets. The *rcv_ping.py* program receives, parses and displays the client's ICMP echo packets. Lab04's programs worked by editing ethernet frames one network layer below via the AF_PACKET family and SOCK_RAW type. These programs work one network layer above via the AF_INET family and SOCK_RAW type.

Run the two programs in two different Docker containers and see if their runtime behavior aligns with your understanding of their code. Note that the programs assume the client and server have certain IP addresses. So, if your containers are assigned different IP addresses, you should adjust these values in the code.

Assuming you stored the programs in the */tmp* folder of your containers, start the server program by typing the following command in the first container:

```
root@container_c1:$ python3 tmp/rcv_ping.py
```

Then start the client program in the second container with the following command:

```
root@container_c2:$ python3 tmp/send_ping.py IP_address_container_c1
```

As you may have realized by playing with the *iputils* ping program in Section 1, a complete implementation of ping on both the client and server sides would require more logic in the respective programs. For example, the client program would need to compute the correct ICMP checksums, manage timeouts for sent requests, and record and update statistics. In Section 4, you will have the chance to incorporate more programming logic into an extended ping client program. In order to do that, however, first you need to learn about an important operating system (OS) level mechanism that will allow you to perform non-blocking I/O operations on sockets in the next section.

¹ Please do not run this command multiple times during the lab, nor increase the indicated number of attempts (i.e., *-c 3*). An unexpected higher number of ping attempts leaving our lab may cause some filtering to be applied to this kind of traffic and affect our ability to run this experiment.

3. Select system call

In POSIX-compliant operating systems, **select** is a system call that allows a program to monitor multiple file descriptors, waiting until one or more of the file descriptors become "ready" for some class of Input/Output (I/O) operation. A file descriptor is considered ready when it is possible to perform a corresponding I/O operation (e.g., reading or writing) without blocking.

The `select` module in Python includes the implementation of the `select()` function and other platform-specific I/O monitoring functions. The `select()` function represents a direct interface to the underlying system call. The main arguments of the `select()` function are three sets of file descriptors that allow the caller to wait for three types of events on the specified set of file descriptors. The `select()` function expects a list of file descriptors to watch for reading, a list of file descriptors to watch for writing, and a list of file descriptors to watch for errors. An optional fourth argument, timeout, can be used to prevent `select()` from blocking indefinitely. `Select()` returns a subset of all three lists in the same order, i.e., all the file descriptors that are ready for reading, writing, or have caused an error. The **select** system call finishes if one of the following conditions occurs:

- one or more of the input channels has data to be read,
- one or more of the output channels are ready to send data,
- an error occurs in one or more of the channels mentioned in the third list,
- timeout expires without any event in the above three sets of channels.

`Select()` can be used to write non-blocking send and receive network requests via sockets. In Python, you use `socket.setblocking(False)` to make a socket non-blocking. This way the `send`, `recv`, `connect` and `accept` can return without having done anything. A program can register multiple sockets, and then constantly call `select()` to see which are the "ready" sockets, effectively handling multiple I/O requests simultaneously within a single thread. The `select` call is blocking, but you can give it a timeout (the fourth input parameter).

The following snippet of code illustrates a simple example of a TCP server that uses the `select()` function to accept multiple client's connections without blocking the main program's control flow.

```
# Example of a server program using select()

import select
import socket
import time

PORT = 12345

s_sck = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s_sck.bind(("", PORT))
s_sck.listen(1)

print("Server listening on port ", PORT)

fd_read = [s_sck]
fd_write = []
fd_error = []
```

```

while True:

    fd_r, fd_w, fd_e = select.select(fd_read, fd_write, fd_error, 1.0)

    if fd_r:
        c_sck, addr = sck.accept()
        # do something with the accepted connection
    else:
        print("Still waiting for connections")

    # run other server's logic without waiting for the "ready" socket
    ...

```

The following code snippet illustrates a simple example of a TCP client that sends and receives messages from a remote server without blocking. This client's behavior is implemented through the use of the `select()` function.

```

# Example of a client program using select()

import socket
import select
import sys

BUFFER_SIZE = 1024

server_IP = ""
client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client_socket.connect((server_IP, 12345))

fd_read = [client_socket]
fd_write = [client_socket]
fd_error = []

while True:

    fd_r, fd_w, fd_e = select.select(fd_read, fd_write, [])

    for s in fd_r:

        # Handle incoming data from the server
        data = s.recv(BUFFER_SIZE)

        if data:
            print(f"Received data from the server: {data.decode()}")
        else:
            print("Server closed the connection")
            sys.exit()

    for s in fd_w:

        # create data message to send
        ...
        s.send(message)

```

4. Exercise - Complete the Pinger client application

Your task is to develop your own ping application in Python. Your application will create and send ICMP echo requests via raw sockets. However, to keep it simple, it will not implement the exact complete behavior of a ping client program. Note that you only need to write the client side of the program because the necessary server-side functionality is built into almost all operating systems. So, you will use that to test your program.

We have provided you with a skeleton of your ping application in the file *pinger_skeleton.py*. Complete the program so that it sends ping requests to a specified host, with a delay of approximately one second between each request. Each ICMP echo request message contains a timestamp in its payload that roughly corresponds to the time at which the packet was generated by the program. After sending each packet, the program waits up to one second for an echo reply. If no reply is received from the host within one second, the program assumes that either the ping or pong packet was lost in the network, or that the specified host is down. If an echo reply is received within the 1-second timeout period, the program parses the ICMP packet to extract the time value in the payload. It then uses this value to determine how long it took to receive a reply to the corresponding ping packet. Typing Ctrl+C (^C) will interrupt the program and print out the computed statistics for all generated packets. Please note that this is a simplified, illustrative version of such a network program that is not intended for precise measurements. Therefore, the printed statistics may differ from those obtained using the classic *ping* utility.

The sections of the program that need to be completed are marked with comment lines that read “*# Fill in - start*” and “*# Fill in - end.*”, and contain additional comments that help you guess the expected program logic to include in each section. One or more lines of code may need to be inserted into these sections to achieve the desired behavior.

Once you have completed the skeleton program by filling in the missing logic, you can test your new program by pinging your host's or your container's loopback interface passing the IP address 127.0.0.1 as an argument to your program.